

Chapter 1: From Operational Databases to Decision Support

Learning Objectives

By the end of this chapter, you should be able to:

1. Explain why an operational (OLTP) database, however fast or well-indexed, is architecturally unsuited to multi-year, cross-departmental analytical questions.
2. Distinguish OLTP and OLAP systems by their actual design priorities, not by speed or hardware alone.
3. Analyze a university Student Information System to identify why a genuine analytical need requires a data warehouse, not a faster version of the same operational database.

A Guided Question to Begin

Picture an IT staff member at a university, asked by the registrar's office why the Student Information System, reliable, fast, and accurate for processing this term's enrollment, can't answer a seemingly simple question: which academic programs have shown declining enrollment over the past five years? The SIS handles today's enrollment perfectly, a student adds a class, the record updates instantly, but the multi-year, cross-department trend question drags for minutes, sometimes timing out entirely.

The IT staff member's first instinct is to treat this as a performance problem: add more indexes to the enrollment table, request a server upgrade, maybe archive old records onto faster storage. After weeks of tuning, indexing every column the trend query touches, even moving the database to considerably more powerful hardware, the query is only marginally faster, still too slow to be genuinely useful, and now every single new-enrollment transaction, the SIS's actual, most frequent job, has gotten measurably slower because of all the new indexes added specifically to speed up a query the system barely ever runs.

Before scrolling further: was this actually a performance problem that more hardware and better indexing should have solved? Or is something more

fundamental going on, something no amount of scaling up the exact same system was ever going to fix?

Introduction

Welcome to Advanced Database Systems. Everything you built in Fundamentals of Database System and Information Management 1, entity-relationship models, normalized relational schemas, SQL that inserts, updates, and retrieves individual records quickly and accurately, was built for a specific job: running an organization's actual, moment-to-moment operations. A database designed this way is called an OLTP system, Online Transaction Processing, and it's optimized around a very particular pattern of work: many short, fast transactions, a single student enrolling in a single class, a single sale being rung up, happening constantly, each one needing to complete correctly and quickly.

This chapter corrects an assumption that feels entirely reasonable the first time an OLTP system is asked a question it wasn't built for: that the problem is simply a matter of speed, and that a faster, bigger, better-indexed version of the same system would eventually get there. It won't, and understanding precisely why is this course's actual starting point.

This chapter uses a university's Student Information System as its running example, tracing exactly why adding indexes and hardware failed to close the gap, and what kind of system actually would.

1.1 What Operational Databases Are Actually Built For

An OLTP system is optimized around many short, fast transactions, each one touching a small number of rows, needing to complete quickly and correctly, often with many transactions happening at once, at different desks, different browsers, different moments, without interfering with each other. The relational, normalized design you practiced in IMFBDS and IM1 serves this job well: normalization reduces redundancy and keeps each individual record consistent, exactly what a system handling constant small updates actually needs.

This is worth stating precisely, because it means the SIS was not poorly designed. It was designed correctly, for the job it was actually built to do.

1.2 The SIS: Fast at Its Real Job, Failing at a Different One

Return to the university's Student Information System, and to the actual gap the IT staff member ran into. The SIS is, by every reasonable measure, an excellent operational database. A student enrolls, the record updates in a fraction of a second. A grade gets posted, it's immediately reflected. This is precisely the job an OLTP system exists to do, and the SIS does it well.

The five-year, cross-department enrollment trend question is a fundamentally different kind of request. It doesn't need one record updated quickly. It needs potentially millions of historical records, across many academic terms and several departments' worth of data, scanned, aggregated, and compared. The SIS was never designed with this access pattern in mind, not because anyone did a poor job designing it, but because designing a single system to handle both jobs equally well is a genuinely difficult, often counterproductive, engineering trade-off (Aerospike, 2026).

1.3 Why More Indexes and Bigger Servers Don't Fix This

It's worth tracing exactly why the IT staff member's instinctive fix, more indexes, a faster server, made things only marginally better, and actually made the SIS's real job worse in the process.

An index speeds up finding a specific, small set of rows quickly, exactly what a transactional lookup needs: is this specific student currently enrolled in this specific class? It does very little to speed up scanning and aggregating a large fraction of an entire table's history, which is what a multi-year trend analysis actually requires. Worse, every index added to a table carries a real, ongoing cost: each new enrollment transaction now has to update not just the enrollment record itself, but every index built on that table, meaning the SIS's actual, most frequent job got measurably slower specifically because of changes made to speed up a query the system rarely runs. Running heavy analytical queries directly against a live transactional system risks exactly this kind of interference, slowing down the operations the system actually exists to support (ThoughtSpot, 2026).

1.4 The Till and the Archive: Two Different Jobs, Two Different Designs

Here's a way to hold what's actually different between these two systems: think of a cashier's daily till compared to a company's archived financial records room.

A till needs to be fast, accurate, and immediately accessible for exactly one kind of task: recording this specific transaction, right now, correctly. Nobody expects to open a cash till and pull out a clear five-year trend of the company's total revenue by product category; that was

never the till's job, and redesigning the till to make that possible would likely make it slower and more cumbersome at the one thing it actually needs to do well, every single day.

An archived records room is organized entirely differently, specifically because its job is different. Records are filed by year, by category, by department, in a structure built for someone to walk in and pull exactly the multi-year comparison they need. Nobody uses the archive to ring up today's sale, and nobody should try; it was never built for that, the same way the till was never built for the archive's job. The university's SIS is the till. The multi-year enrollment trend question needs something that works like the archive, a data warehouse, not a faster, bigger version of the till.

Pause and reflect: Think of an operational system you use or have observed, a school portal, a store's checkout, a clinic's booking system. What is one question about its data, across a long time span or several categories at once, that system would genuinely struggle to answer quickly, no matter how well it's maintained?

1.5 What a Data Warehouse Actually Is

A data warehouse is not a backup copy of the operational database, and it's not simply the same schema running on more powerful hardware. It's a separate system, deliberately organized around a different access pattern: OLAP, Online Analytical Processing, built specifically to support scanning, aggregating, and comparing large volumes of historical data across time, department, or any other dimension a real decision might need.

Figure 1 lays the two systems' actual design priorities side by side, since the contrast is really the entire point of this chapter.

Figure 1. OLTP (the SIS) vs. OLAP (a Data Warehouse): Different Jobs, Different Designs

OLTP		OLAP
the Student Information System		a data warehouse built for analysis
Many fast, small transactions	Primary job	Few large, analytical queries
Current, operational data	Data scope	Historical data across time
"Is this student enrolled in this class right now?"	Typical question	"How has enrollment changed by department over 5 years?"
Fast writes (insert / update)	Optimized for	Fast reads across large datasets
Row-based	Storage pattern	Often column-based
Normalized (reduces redundancy)	Schema design	Denormalized / dimensional (star or snowflake schema)

Figure 1. OLTP and OLAP are optimized for genuinely different jobs: many fast small transactions versus a small number of large analytical scans.

Notice that none of these rows is about one system being “better” than the other. OLTP systems typically use row-based storage, efficient for updating a full record at once, while OLAP systems commonly use column-based storage, efficient for scanning just the specific fields a large aggregation actually needs (VeloDB, 2026). They're optimized for genuinely different jobs, the same distinction separating a till from an archive, and an organization that actually needs both, fast daily operations and genuine historical insight, needs both kinds of system, not one system stretched to do both badly.

1.6 Where This Course Goes From Here

This chapter's correction is the foundation the rest of this course builds on directly. Having established that the actual gap is architectural, not a matter of scale, the next several chapters walk through exactly how a data warehouse is actually built: its architecture, the ETL process that moves and reshapes data from operational systems like the SIS into it, the dimensional modeling that gives it its distinctive, scan-friendly structure, and the OLAP operations that let

someone actually ask, and get real answers to, exactly the kind of question the SIS was never built to answer.

If your OLIMFBDS case study involved a scenario like the university's enrollment system, or something structurally similar, you already have exactly the right starting material this course will keep building on. The database you designed there was built correctly, for the job it was actually meant to do. This course is about building the different, complementary system sitting beside it.

Common Misunderstandings to Watch For

Each item below reflects a version of this chapter's central lesson, applied to a slightly different situation. Reviewing them now is good practice for this week's assessment.

1. Assuming a slow analytical query on an operational database means the database needs more indexes or a bigger server, rather than recognizing an architectural mismatch.
2. Adding indexes aimed at a rarely run analytical query without weighing the real, ongoing cost those same indexes add to the system's actual, frequent transactions.
3. Assuming a data warehouse is simply a backup or a faster copy of the operational database, rather than a separately designed system built for a different access pattern.
4. Treating OLTP and OLAP as a hierarchy, one “better” than the other, rather than recognizing both are genuinely necessary for different jobs.

Key Terms Introduced This Chapter

OLTP (Online Transaction Processing): A database system optimized for many short, fast transactions, typical of operational systems like a Student Information System.

OLAP (Online Analytical Processing): A database system optimized for scanning, aggregating, and comparing large volumes of historical data to support analysis and decisions.

Data warehouse: A separately designed system, built around OLAP access patterns, that integrates and organizes historical data specifically to support analytical questions.

Summary

Go back to that IT staff member one more time: weeks spent adding indexes, requesting a faster server, and a trend query still too slow to be useful, while the SIS's actual, everyday job of

enrolling students got measurably slower in the process. You now have language for why that effort was never going to fully succeed. The problem was never how fast the SIS ran. It was what the SIS was actually designed to do, quick, accurate, individual transactions, and no amount of scaling changes a system's fundamental design purpose.

You can also say, now, why a till and an archive are both genuinely necessary, and why neither one should try to become the other. The SIS's job, fast, reliable, moment-to-moment operations, remains exactly as important as it always was. The five-year enrollment trend question simply needs a different kind of system, built with a different job in mind from the start.

Carry this distinction into every chapter ahead. This course isn't about replacing what you built in IMFBDS and IM1. It's about building the second system, the archive standing beside the till, purpose-built for the questions an operational database was never meant to answer.

Performance Task

Title: Diagnosing the Gap: Performance Problem or Architecture Problem?

For this task, you will identify a real analytical question an operational database would struggle to answer, and explain why more hardware or indexing would not genuinely solve it.

Your submission should include the following parts:

5. Identify one real or realistic organizational scenario (this may be your own OLIMFBDS case study organization) with an operational database that handles day-to-day transactions well.
6. Propose one specific multi-year or cross-departmental analytical question this operational database would struggle to answer, and explain specifically why, using this chapter's OLTP versus OLAP distinction rather than simply "it would be slow."
7. In 150 to 200 words, explain why adding indexes or upgrading hardware alone would not adequately solve this problem, and what the underlying architectural mismatch actually is.
8. In 100 to 150 words, briefly describe what a data warehouse serving this analytical question might need to look like, in general terms. This will be developed further in later chapters.

Submit your complete analysis through the LMS by the stated deadline.

Note: this task is evaluated on the accuracy of your architectural reasoning, not on how large or complex your chosen organization is. A small, ordinary operational database with a genuine analytical gap is exactly what this task is asking for.

References

Aerospike. (2026, May 20). OLTP vs. OLAP explained. <https://aerospike.com/blog/oltp-vs-olap/>

ThoughtSpot. (2026, May 27). OLAP vs OLTP: Key differences explained.

<https://www.thoughtspot.com/data-trends/data-modeling/oltp-vs-olap>

VeloDB. (2026, March 21). OLAP database: Definition, OLAP vs OLTP & best tools.

<https://www.velodb.io/glossary/olap-database>